Описание процессов жизненного цикла

Документ описывает, как сопровождается программа для ЭВМ «XML-библиотека „Берёста“ (beresta)»: где хранится и собирается исходный текст, как выпускаются версии, как принимаются и устраняются сообщения об ошибках и уязвимостях, какие версии поддерживаются и кто обеспечивает поддержку.

Правообладатель: [ФИО правообладателя]. Контакт для вопросов о сопровождении: [адрес электронной почты].

Содержание

1. Общие сведения
2. Хранение исходного текста и сборка
3. Версии и выпуск
4. Поддерживаемые версии
5. Сообщения об ошибках
6. Уязвимости
7. Контроль качества
8. Совершенствование
9. Персонал, обеспечивающий поддержку

Общие сведения

Параметр	Значение
Вид продукта	библиотека для обработки XML (встраивается в прикладные программы)
Лицензия	MIT; программа предоставляется на безвозмездной основе
Язык разработки	C++17, без сторонних зависимостей
Репозиторий исходного текста в России	Gitverse: <https://gitverse.ru/ermaleyalekov/beresta>
Зеркало репозитория	GitHub: <https://github.com/ErmaleyAlekov/beresta>
Журнал изменений	файл `CHANGELOG.md` в корне репозитория
Средства активации и лицензионные ключи	не используются: библиотека не требует активации, ключей, регистрации и подключения к сети
Обновление	только по действию пользователя (новая версия скачивается и подключается разработчиком программы); принудительного или удалённого обновления нет

Хранение исходного текста и сборка

- Исходный текст хранится в системе контроля версий git. Основная ветка — `main`, каждая выпущенная версия отмечена тегом `vX.Y.Z`. История изменений доступна полностью.
- Репозиторий размещён на площадке Gitverse (Россия) и зеркалируется на GitHub; изменения отправляются на обе площадки одновременно.
- Сборка воспроизводима из исходного текста стандартными средствами (GNU Make или CMake и компилятор C++17), без загрузки чего-либо из сети. Проверочный стенд `tools/stand/run_stand.sh` выполняет полную сборку и проверку одной командой.
- Непрерывная интеграция: на Gitverse (`.gitverse/workflows`) при каждом изменении выполняются сборка и модульные тесты на Linux x86_64; на GitHub (`.github/workflows`) — полная матрица платформ (Linux x64 и arm64, Windows x64 и arm64, macOS, WebAssembly), санитайзеры и W3C XML Conformance Test Suite.
- Сборка выпусков: [указать, где выполняется сборка выпускаемых файлов для реестра — например, на Gitverse или на компьютере правообладателя в России].

Версии и выпуск

Номер версии — `MAJOR.MINOR.PATCH` (семантическое версионирование):

- `PATCH` — исправления ошибок без изменения программного интерфейса;
- `MINOR` — новые возможности с сохранением совместимости;
- `MAJOR` — изменения, нарушающие совместимость (описываются в журнале изменений вместе со способом перехода).

Порядок выпуска версии:

1. Версия поднимается во всех файлах сборки и пакетов, в `CHANGELOG.md` добавляется раздел версии.
2. Выполняются модульные тесты (`make test`) — все должны пройти; для изменений парсера и XML Schema дополнительно прогоняются наборы W3C, процент прошедших случаев не должен уменьшаться.
3. Изменения отправляются в репозитории Gitverse и GitHub; CI на обеих площадках должен завершиться успешно.
4. Ставится тег `vX.Y.Z`; CI собирает бинарники для всех платформ, пакеты (npm, PyPI, NuGet, Maven Central) и архивы SDK, проверяет их в чистом окружении и публикует релиз.
5. Тот же релиз с теми же файлами публикуется на Gitverse.

История выпусков:

Версия	Дата	Содержание
2.1.2	2026-10-20	проверка XML Schema по W3C XML Schema Test Suite: 93,6 % → 96,2 % применимых случаев; релизы на Gitverse

Версия	Дата	Содержание
2.1.1	2026-10-19	см. описание релиза
2.1.0	2026-10-19	см. описание релиза

Подробное содержание каждой версии — в `CHANGELOG.md`.

Частота выпусков: исправления (`PATCH`) выходят по мере устранения ошибок, как правило не позднее [N] дней после подтверждения ошибки; версии с новыми возможностями (`MINOR`) — по мере готовности, ориентировочно [N] раз в год. [Правообладателю — уточнить сроки.]

Поддерживаемые версии

Версия	Поддержка
2.1.x	исправления ошибок и уязвимостей
младше 2.1	не поддерживаются, рекомендуется обновиться до 2.1.x

Исправления выпускаются в последней минорной ветке. Совместимость Си-интерфейса (имена функций, коды состояния, номера сообщений) сохраняется в пределах мажорной версии.

Сообщения об ошибках

Сообщить об ошибке можно в разделе «Задачи» репозитория на Gitverse или в Issues на GitHub:

- Gitverse: <https://gitverse.ru/ermaleyalekov/beresta/tasktracker>
- GitHub: <https://github.com/ErmaleyAlekov/beresta/issues>

В сообщении укажите:

- версию «Берёсты» (`beresta_version()` или `BERESTA_VERSION_STRING`), ОС, компилятор или язык обёртки;
- минимальный XML, XSD или выражение XPath, на котором ошибка повторяется, ожидаемый и фактический результат;
- для ошибок соответствия стандарту — пункт спецификации (XML 1.0, Namespaces, XSD 1.0 / 1.1, XPath 1.0) или случай набора тестов W3C.

Порядок устранения:

1. Регистрация: сообщение получает ответ и метку (ошибка, вопрос, предложение) — в срок до [N] рабочих дней.
2. Воспроизведение: ошибка повторяется на указанной версии; при необходимости запрашиваются данные.

3. Исправление: в исходный текст вносится исправление, а в модульные тесты (`tests/src/<модуль>/`) — тест, который до исправления не проходил.
4. Проверка: модульные тесты, при необходимости наборы W3C; CI на обеих площадках.
5. Выпуск: исправление входит в ближайшую патч-версию и описывается в `CHANGELOG.md`; задача закрывается со ссылкой на версию.

Уязвимости

Порядок описан в файле `SECURITY.md` репозитория.

- Об уязвимости сообщают закрыто: через GitHub (Security → Report a vulnerability) или, без подробностей, задачей на Gitverse с просьбой о закрытом канале связи. Публичные задачи с подробностями уязвимости не открываются.
- Подтверждение получения — в течение 7 дней, оценка и план исправления — в течение 30 дней.
- Исправление выходит патч-версией на Gitverse и GitHub и в пакетах всех языков, с описанием в `CHANGELOG.md`.
- Уязвимостью считается, в частности, загрузка внешних ресурсов без явного разрешения (XXE, SSRF), обход лимитов сущностей и глубины, нелинейное время обработки, падение, выход за границы памяти или неопределённое поведение на любом входном документе, схеме или выражении XPath.

Контроль качества

- Модульные тесты C++ (`make test`), в том числе нагрузочные и многопоточные.
- W3C XML Conformance Test Suite — 100 % применимых случаев; W3C XML Schema Test Suite — 96,2 % применимых случаев (расхождения перечислены в `tests/xsts`).
- Проверка санитайзерами AddressSanitizer, UndefinedBehaviorSanitizer, LeakSanitizer и ThreadSanitizer; фаззинг libFuzzer (разбор, XPath, XML Schema).
- Единые правила кода: короткие функции, небольшие файлы, двуязычные комментарии (`CONTRIBUTING.md`).

Совершенствование

Развитие ведётся по задачам в репозиториях. Текущие направления:

- повышение соответствия W3C XML Schema Test Suite (проверка документа схемы по схеме для схем);
- новые возможности XSD 1.1 и расширение Си-интерфейса по запросам пользователей;
- проверка сборки на российских операционных системах (Astra Linux, РЕД ОС, ALT) и процессорах.

Предложения принимаются в задачах Gitverse и GitHub; внешние изменения — через запросы на слияние по правилам `CONTRIBUTING.md`.

Персонал, обеспечивающий поддержку

ФИО	Роль	Квалификация	Место нахождения
[ФИО]	разработчик и сопровождающий: исправление ошибок, выпуск версий, ответы на сообщения	[образование, опыт]	Российская Федерация, [город]
[ФИО, если есть]	[роль]	[квалификация]	[город]

Техническая поддержка оказывается [на русском языке, бесплатно, через задачи Gitverse и по электронной почте [адрес]].